

Experimental Evaluation and Measurements

1- Compute the average response time (query/buy) of your new systems. What is the response time with and without caching? How much does caching help?

This is the timing analysis of response time for **getInfo/{id} query** What is the response time with and without caching? How much does caching help?

	Without caching	With caching
getInfo/3	486 ms	6 ms
getInfo/10	287 ms	5 ms
getInfo/64	64 ms	8 ms
getInfo/30	33 ms	5 ms
getInfo/6	19 ms	5 ms
getInfo/7	18 ms	4 ms
Avg.	151.66 ms	5.5 ms

Note : The results are taken from Postman in addition to our code timing snippets. Therefore, they may not be perfectly identical, but there are only slight differences between them.

Response Time : time from where you made the request until you receive the corresponds response

Discussion :

The table clearly demonstrates the impact of implementing caching on system performance. Without caching, the response times vary significantly, This occurs since we have a **cache miss** because the system must retrieve data directly from the database or process the request fully each time so this is what actually occurs :

- The system must retrieve the data directly from the database or backend service.
- Additional processing, network communication, and data fetching ... are required.
- After retrieving the data, the system stores it in the cache for future use.

After enabling caching, the response times become much lower and more stable for each request , ranging between **4–8 ms**. This indicates that repeated requests are being served directly from the cache memory (**in-memory cache which is in our case HashMap**) instead of querying the database again, which greatly reduces processing overhead.

The average response time without caching is approximately **151.66 ms**, while with caching it drops to only **5.5 ms**. This shows a major improvement in efficiency and highlights the importance of caching in reducing latency and improving user experience.

Main point to get with it , that in first 2 requests the timing analysis without caching are extremely high ranged between 300-400ms even in other requests even there is a cache miss but they are extremely fewer compared with the first ones , this is can be proved with the following :

Such as these cases we call them as **Cold Start** as we say there is cache miss ... , in addition of that , for the initial requests the JVM and many process in the background related to data base , establishing the TCP connection , networking are still in warming up stage so in other words there is an external background process .

The next requests behave as **Database Native Caching (≈ 50 ms)**: By the third request even that each request will set up his database connection and once the request is finished then the connection will be broken since we use try-with-resources , still the connection pool is warmed up and the database has cached its query execution plans internally .

The rest requests behave as **Application-Level Caching (≈ 5.5 ms)**: With caching enabled, you bypass the database network hop entirely. The data is served directly from RAM (like an in-memory `HashMap`), dropping response times by 97% and maintaining a flat, predictable 5 ms average.

2- Construct a simple experiment that issues orders or catalog updates (i.e., database writes) to invalidate the cache and maintain cache consistency. What are the overhead of cache consistency operations? What is the latency of a subsequent request that sees a cache miss?

The following is related to order service :

1- getOrders/ : cache miss : 773 ms

2- getOrders/ : cache hit : 9 ms

3-makeOrder/64/2: invalidation occurred (no caching in our code since it is write operation)
752 ms

4-getOrders/: cache miss : 50 ms

5- getOrders/ : cache hit : 12 ms

6-makeOrder/64/4: invalidation occurred (no caching in our code since it is write operation)
84 ms

7-getOrders/: cache miss : 33 ms

8- getOrders/ : cache hit : 8 ms

Discussion :

When the `makeOrder/64/2` operation is executed, cache invalidation occurs because the database content has changed and the cached order data is no longer valid. Since write operations are not cached in the implementation, the request directly updates the database and removes the outdated cached data. This operation takes **752 ms**, which represents the overhead of maintaining cache consistency.

After invalidation, the next `getOrders/` request experiences a cache miss again with a latency of **50 ms** because the cache must be rebuilt from the updated database state. Once the new data is cached, the subsequent request becomes a cache hit and the latency decreases again to **12 ms**.

The same behavior appears after the second write operation `makeOrder/64/4`, where invalidation occurs with a latency of **84 ms**. The following read request again becomes a cache miss with **33 ms**, followed by a cache hit of only **8 ms**.

These results show that cache consistency operations introduce additional overhead because the system must:

- Remove outdated cached data.
- Re-fetch updated data from the database.
- Rebuild the cache after write operations.

However, despite this overhead, caching still provides substantial performance improvements for repeated read operations. The latency of requests after cache invalidation is noticeably higher than cache hits, but still lower than the very first request due to server warm-up and already initialized resources.

Our invalidation functionality :

```
private static void invalidateOrderCache() {  
  
    orderCache.clear();  
}
```